USEI06 – Time-Zone Index and Windowed Queries

Context

Railway stations must be efficiently queryable by:

- **Time zone group and country**
- **Geographic coordinates (latitude/longitude ranges)**
- **Multiple time zone groups (windowed queries)**

These operations require fast lookups without scanning all 64k stations sequentially.

User Story

As a planner, I want a set of BST/AVL trees that allows me to quickly search by latitude, longitude, time zone group, and country, so I can efficiently retrieve stations in ascending order.

Acceptance Criteria

- Validate: non-empty station name, country, and time zone group.
- Validate: latitude [-90, 90], longitude [-180, 180].
- Reject invalid rows during CSV import.
- If multiple stations share the same coordinates (e.g., **Lisbon Santa Apolónia** and **Lisbon Oriente** at 38.71387, -9.122271), they must be stored together and sorted alphabetically by station name.
- The output must include:
  - 3–5 sample query results
  - A temporal complexity analysis

Temporal Complexity Analysis

1. Index Construction

The index is built in two phases:

Phase 1: Adding Stations to TreeMaps

```
public void addStation(Station station) {
    // Add to latitude index
```

```
    latitudeIndex.computeIfAbsent(station.getLatitude(), k -> new ArrayList<>())
            .add(station);                                          // O(log N)

    // Add to longitude index
    longitudeIndex.computeIfAbsent(station.getLongitude(), k -> new ArrayList<>())
            .add(station);                                          // O(log N)

    // Add to timeZone index (TimeZoneGroup -> Country -> Stations)
    timeZoneIndex.computeIfAbsent(station.getTimeZoneGroup(), k -> new TreeMap<>())
            .computeIfAbsent(station.getCountry(), k -> new ArrayList<>())
            .add(station);                                          // O(log K + log C)

    totalStations++;
}
```

- **TreeMap insertions**: O(log N) per station for each index
- **For N stations**: O(N log N)
- Where:
  - N = total number of stations (~64k)
  - K = number of unique time zone groups
  - C = number of unique countries per time zone

Phase 2: Finalizing the Index (Sorting Duplicates)

```
public void finalizeIndex() {
    // Sort latitude index stations by name
    for (List stations : latitudeIndex.values()) {
        Collections.sort(stations);                                 // O(M log M)
    }

    // Sort longitude index stations by name
    for (List stations : longitudeIndex.values()) {
        Collections.sort(stations);                                 // O(M log M)
    }

    // Sort timeZone index stations by name
    for (TreeMap> countryMap : timeZoneIndex.values()) {
        for (List stations : countryMap.values()) {
            Collections.sort(stations);                             // O(M log M)
        }
    }
}
```

- **Sorting each bucket**: O(M log M), where M = max stations sharing coordinates
- **Total buckets**: O(U), where U = unique coordinate pairs
- **Overall finalization**: O(U × M log M)

Total Construction Complexity: **O(N log N + U × M log M)**

In practice, M is small (typically 1-3 stations per coordinate), so:
O(N log N)

---

2. Query Operations

Query by Time Zone Group

```
public List getStationsByTimeZoneGroup(String timeZoneGroup) {
    TreeMap> countryMap = timeZoneIndex.get(timeZoneGroup); // O(log K)
    if (countryMap == null) {
        return Collections.emptyList();
    }

    List result = new ArrayList<>();
    for (List stations : countryMap.values()) {    // O(C)
        result.addAll(stations);                              // O(R)
    }
    return result;
}
```

- **TreeMap lookup**: O(log K)
- **Collecting results**: O(R), where R = number of result stations
- **Overall**: **O(log K + R)**

Query by Time Zone Group and Country

```
public List getStationsByTimeZoneGroupAndCountry(String timeZoneGroup, String country) {
    TreeMap> countryMap = timeZoneIndex.get(timeZoneGroup); // O(log K)
    if (countryMap == null) {
        return Collections.emptyList();
    }
    List stations = countryMap.get(country);       // O(log C)
    return stations != null ? new ArrayList<>(stations) : Collections.emptyList();
}
```

- **Overall**: **O(log K + log C + R)**

Query by Time Zone Window

```
public List getStationsByTimeZoneWindow(String... timeZoneGroups) {
    List result = new ArrayList<>();
    Arrays.sort(timeZoneGroups);                              // O(W log W)

    for (String tzGroup : timeZoneGroups) {                   // O(W)
        result.addAll(getStationsByTimeZoneGroup(tzGroup)); // O(log K + R_i)
```

```
    }
    return result;
}
```

- **Sorting time zones**: O(W log W), where W = number of time zone groups
- **Fetching results**: O(W × (log K + R_avg))
- **Overall**: **O(W log W + W log K + R_total)**

Range Queries (Latitude/Longitude)

```
public List getStationsByLatitudeRange(double minLat, double maxLat) {
    List result = new ArrayList<>();

    for (Map.Entry> entry :
            latitudeIndex.subMap(minLat, true, maxLat, true).entrySet()) { // O(log N + R)
        result.addAll(entry.getValue());
    }
    return result;
}
```

- **TreeMap.subMap**: O(log N) to find range boundaries
- **Collecting R results**: O(R)
- **Overall**: **O(log N + R)**

Summary Table

Operation	Expected Complexity	Notes
Index Construction	O(N log N)	N stations, balanced TreeMap insertions
Index Finalization	O(U × M log M) O(N)	U unique coords, M small (1-3 typically)
Query by Time Zone Group	O(log K + R)	K time zone groups, R results
Query by TZ + Country	O(log K + log C + R)	C countries per time zone
Query by TZ Window	O(W log W + W log K + R)	W time zones queried
Range Query (Lat/Lon)	O(log N + R)	Efficient subMap operation
Memory Complexity	O(N)	Three TreeMap indices